WHO WANTS TO BE KING

VulnHub CTF — MACHINE

Lab Environment

Attacker Machine: Kali Linux
Attacker IP: 192.168.56.104

Target Machine: Who Wants To Be King
Target IP: 192.168.56.101

FROM

Saniya Shaikh

Contents

1. Objective 3
2. Lab Environment 3
3. Target Information 3
4. Reconnaissance 3
Step 1 — Identifying the Kali (Attacker) IP Address 3
Step 2 — Discovering Live Hosts with arp-scan 4
Step 3 — Scanning Both Candidate Hosts for Open Ports and Services 5
5. Web Enumeration 6
Step 4 — Viewing the Target's Web Directory in a Browser 6
Step 5 — Downloading the skeylogger Binary 6
6. Binary Analysis and Credential Discovery 7
Step 6 — Inspecting skeylogger for Embedded Strings 7
Step 7 — Decoding the Embedded Base64 String 8
7. Initial Access 9
Step 8 — Logging In Over SSH as daenerys 9
8. Local Enumeration as daenerys 10
Step 9 — Listing the Home Directory 10
Step 10 — Investigating ~/.local/share 11
Step 11 — Examining daenerys.zip and the djkdsnkjdsn Pointer File 12
Step 12 — Reading the Referenced Clue File 13
9. OSINT-Based Clue Research 13
Step 13 — Researching the Term "Khal" 13
Step 14 — Identifying the Specific Khal: Khal Drogo 14
10. Privilege Escalation to Root and Root Access and Final Flag 15
Step 15 — Escalating to Root with su 15
12. Credentials and Artifacts Summary 16
13. Conclusion 17

1. Objective

The goal of this exercise was to gain a foothold on a Game of Thrones themed VulnHub-style boot2root machine, "Who Wants To Be King," and escalate privileges from an unauthenticated network position all the way to root. Unlike a straightforward exploit-based box, this machine leans heavily on reconnaissance, binary inspection, and narrative-driven clues that had to be researched and interpreted correctly at each stage. This report documents every command that was actually run, in the exact order it happened, along with what each result meant and why it led to the next action taken.

2. Lab Environment

Attacker machine: Kali Linux (user: saniya)

Target machine: "Who Wants To Be King" boot2root VM

Network: isolated host-only/NAT network 192.168.56.0/24, interface eth1 on Kali

3. Target Information

Item	Value
Attacker IP (Kali)	192.168.56.104
Other host on network	192.168.56.100 (all 65,535 ports filtered — not the intended target)
Target IP	192.168.56.101
Open ports	22/tcp (SSH), 80/tcp (HTTP)
Web server	Apache httpd 2.4.41 (Ubuntu)
Notable web content	Open directory listing exposing a file named "skeylogger"

4. Reconnaissance

Reconnaissance is the first stage of any penetration test. The goal here was to confirm the attacker's own network identity, discover which hosts were alive on the lab network, and enumerate the services running on the target so the attack surface was clear before anything else was attempted.

Step 1 — Identifying the Kali (Attacker) IP Address

Command:

```
ip a
```

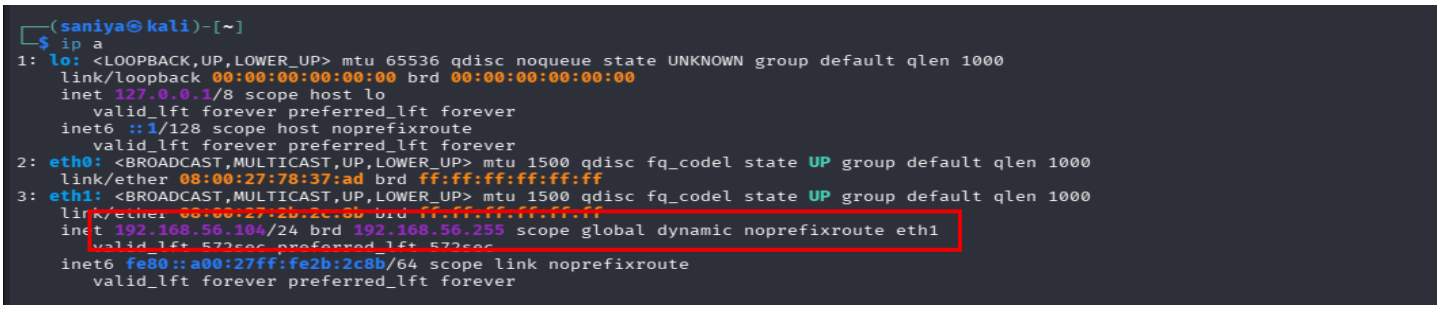


Figure 1: Kali Network Configuration (ip a)

Description: The ip a command lists every network interface on the Kali machine along with its assigned IP address. It was run first to confirm exactly which interface and address belonged to the attacker before any scanning began.

Result: Interface eth1 was assigned the address 192.168.56.104/24, with the MAC address 08:00:27:2b:2c:8b. The loopback (lo) and eth0 interfaces were also listed but were not relevant to the lab network.

Interpretation: 192.168.56.104 is the Kali (attacker) machine's own IP address on the isolated lab network. Any target machine for this challenge must sit on the same 192.168.56.0/24 subnet.

Why this matters: Knowing the attacker's own IP up front avoids confusing attacker and target traffic later, and is required if a reverse shell or listener is ever needed.

Next step: With the attacker's own address confirmed, the next task was to sweep the 192.168.56.0/24 range to discover which other hosts were alive.

Step 2 — Discovering Live Hosts with arp-scan

Command / Action:

```
sudo arp-scan --interface=eth1 --localnet
```

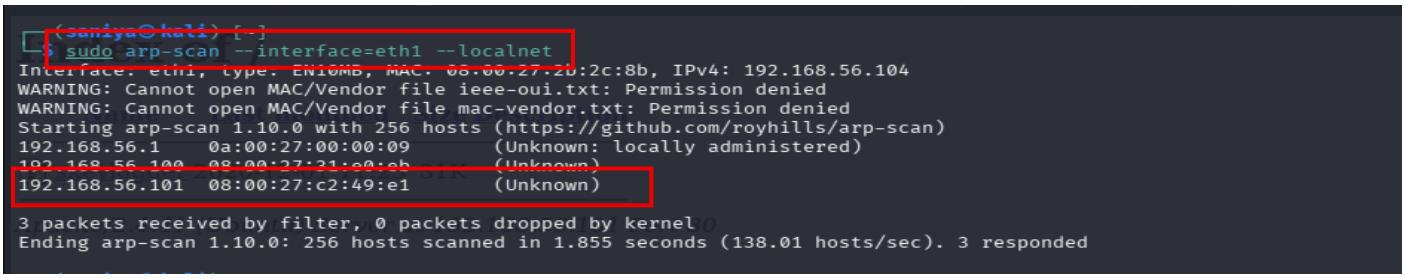


Figure 2: arp-scan Host Discovery Results

Description: arp-scan sends ARP requests to every address on the local subnet and lists any host that replies. It was pointed at eth1's local network to discover every device sharing the lab segment with Kali.

Result: Three hosts responded: 192.168.56.1 (the virtual network gateway), 192.168.56.100, and 192.168.56.101. All three were reported as unknown/locally administered MAC vendors, which is typical for VirtualBox host-only adapters.

Interpretation: Two candidate target machines were now known: .100 and .101. Both needed to be scanned for open ports before either could be ruled in or out as the actual challenge target.

Why this matters: Without this step, the correct target IP would have had to be guessed rather than confirmed, and time could easily have been wasted attacking the wrong host.

Next step: Both candidate hosts, .100 and .101, were scanned together with Nmap to identify open ports and running services.

Step 3 — Scanning Both Candidate Hosts for Open Ports and Services

Command / Action:

```
sudo nmap -sV -sC -p- 192.168.56.100 192.168.56.101
```

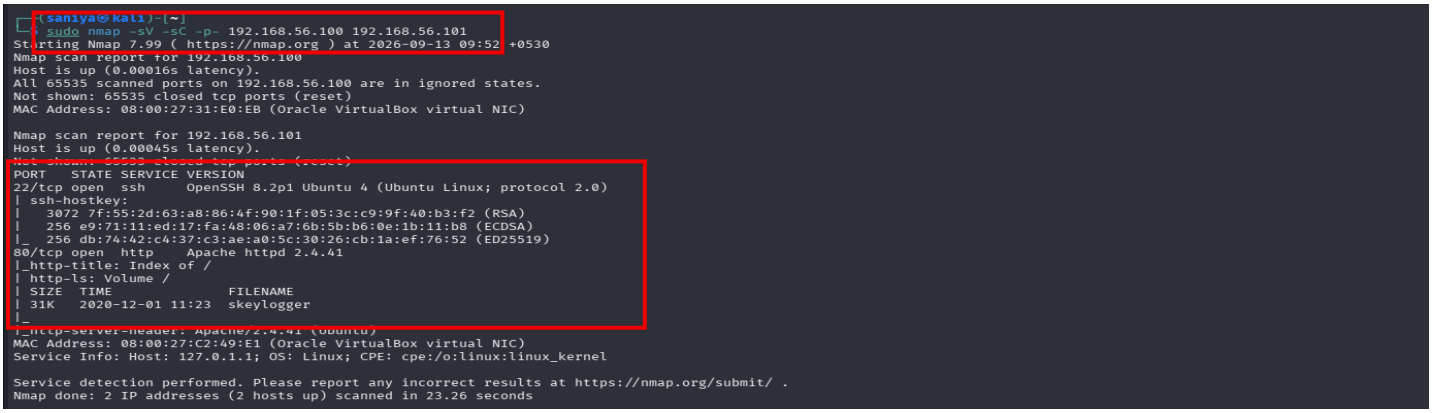


Figure 3: Nmap Full Port and Service/Script Scan Results

Description: The -sV flag detects service versions, -sC runs Nmap's default enumeration scripts, and -p- scans all 65,535 TCP ports. Both discovered hosts were scanned in a single command to compare them directly.

Result: 192.168.56.100 returned no open ports at all — every port was in an ignored/closed state, meaning it was not offering any usable service. 192.168.56.101 returned two open ports: **22/tcp running OpenSSH 8.2p1 on Ubuntu**, **and 80/tcp running Apache httpd 2.4.41**. The HTTP script output also included an http-ls listing showing a single file**, skeylogger, sized 31K**, sitting in the web root.

Interpretation: 192.168.56.101 is clearly the intended target: it exposes both SSH and a web server, and the web server is directly listing a downloadable file. 192.168.56.100 could be safely set aside since it offered no attack surface at all.

Why this matters: This scan did two jobs at once: it identified the real target, and it produced the very first concrete lead — a mysterious binary called skeylogger sitting in an open web directory.

Next step: The target's web server was opened in a browser to confirm the directory listing and examine the skeylogger file directly.

5. Web Enumeration

With port 80 confirmed open and an interesting filename already spotted by Nmap, the website itself became the next stop. The goal here was simply to confirm what was being served and retrieve the file for offline analysis.

Step 4 — Viewing the Target's Web Directory in a Browser

Command / Action:

```
Browsed to http://192.168.56.101
```

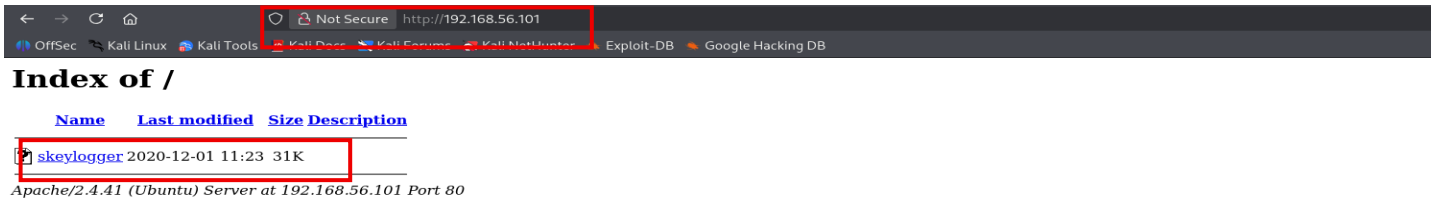


Figure 4: Apache Directory Listing of the Target Web Root

Description: The target's root URL was opened directly in the browser to see what Apache was serving.

Result: Apache returned a plain "Index of /" directory listing (no custom homepage exists) containing a single entry: skeylogger, last modified 2020-12-01, 31K in size.

Interpretation: This confirmed exactly what the earlier Nmap http-ls script had already reported. The web server's only purpose here is to host this one file for download — there is no application to attack on port 80 beyond retrieving it.

Why this matters: It confirmed the next action should be to pull the file down to Kali and inspect it locally, rather than continuing to probe the web server itself.

Next step: The skeylogger file was downloaded to the attacker machine with wget.

Step 5 — Downloading the skeylogger Binary

Command / Action:

```
wget http://192.168.56.101/skeylogger
```

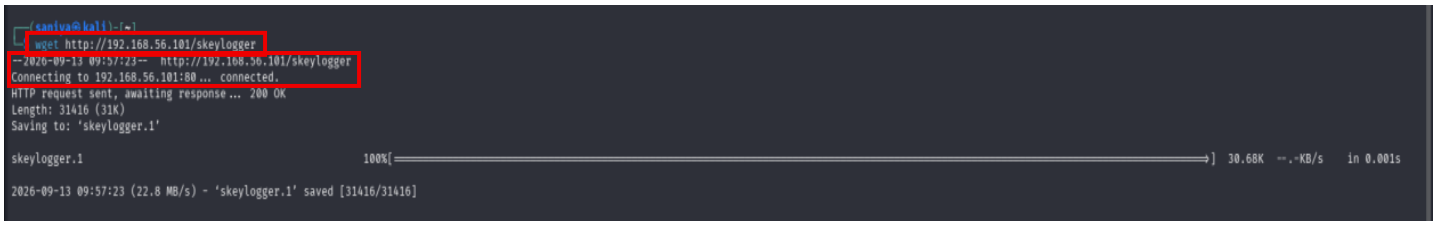


Figure 5: Downloading skeylogger with wget

Description: wget was used to fetch the file directly from the web server and save a local copy for analysis.

Result: The server responded 200 OK and the file was saved locally as skeylogger.1, 31,416 bytes, matching the size shown earlier by Nmap and the browser.

Interpretation: The download completed cleanly, confirming the file was reachable and intact. Its name strongly suggests it is a compiled keylogging utility rather than a document or script.

Why this matters: Having a local copy meant the binary could be executed and inspected safely and repeatedly without depending on the web server.

Next step: The binary was run/inspected on Kali to understand its behaviour and check for anything unusual hidden inside it.

6. Binary Analysis and Credential Discovery

Step 6 — Inspecting skeylogger for Embedded Strings

Command:

```
chmod +x skeylogger
./skeylogger --help
strings skeylogger
```

```
<KP/>
<SysRq>
<RAlt>
<Home>
<Up>
<PageUp>
<Left>
<Right>
<End>
<Down>
<PageDown>
<Insert>
<Delete>
Unknown key: %u
hvl:d:
help
version
logfile
device
Could not determine keyboard device file
ZHJhY2FyeXMK
Usage: skeylogger [OPTION]
Logs pressed keys
  -h, --help            Displays this help message
  -v, --version         Displays version information
  -l, --logfile         Path to the logfile
  -d, --device          Path to device file (/dev/input/eventX)
Simple Key Logger version 0.0.1
grep -E 'Handlers|EV' /proc/bus/input/devices |grep -B1 120013 |grep -Eo event[0-9]+ |tr '\n' '\0'
;*3$"
GCC: (Debian 10.2.0-16) 10.2.0
        val
/usr/lib/gcc/x86_64-linux-gnu/10/include
```

Figure 6: skeylogger Help Output and Embedded Strings

Description: The binary's help output was reviewed first, showing it is a "Simple Key Logger version 0.0.1" that reads raw input events from /dev/input and logs pressed keys. Because it is a small, custom-built utility for this specific machine, its printable strings were also reviewed for anything the developer may have left behind, such as debug data, comments, or hard-coded values.

Result: Among the normal help text, usage flags, and a grep/tr one-liner the tool uses internally to locate the correct input device, one line stood out from everything else: the string **ZHJhY2FyeXMK**. Every other string was ordinary help text or build metadata (GCC version, include paths); this one had the unmistakable look of Base64.

Interpretation: A random-looking alphanumeric string sitting in the middle of an otherwise ordinary keylogger's strings output, ending in the classic "="-free but block-aligned Base64 pattern, is a strong signal that the machine's author hid a clue inside this binary deliberately.

Why this matters: This was the second concrete lead on the box — a possible encoded credential or password hint planted inside the only file the web server exposed.

Next step: The suspicious string was decoded locally with base64 -d to see what it actually contained.

Step 7 — Decoding the Embedded Base64 String

Command :

```
echo -n 'ZHJhY2FyeXMK' | base64 -d
```



Figure 7: Decoding the Hidden String — Result: dracarys

Description: The candidate Base64 string pulled from the skeylogger binary was decoded on the command line.

Result: The decoded value was **dracarys** — the Valyrian command Daenerys Targaryen gives her dragons in the Game of Thrones universe, meaning "fire."

Interpretation: This confirmed the machine's Game of Thrones theme and, more importantly, produced a very plausible password. Since SSH was the only other open service on the target, and the theme points toward a Targaryen-related account, "dracarys" was the natural password to try for an SSH login.

Why this matters: This turned a binary that looked like a red herring into the actual credential needed for initial access to the box.

Next step: SSH access to the target was attempted using the decoded password against a Game of Thrones themed username.

7. Initial Access

Step 8 — Logging In Over SSH as daenerys

Command :

```
ssh daenerys@192.168.56.101
Password: dracarys
```

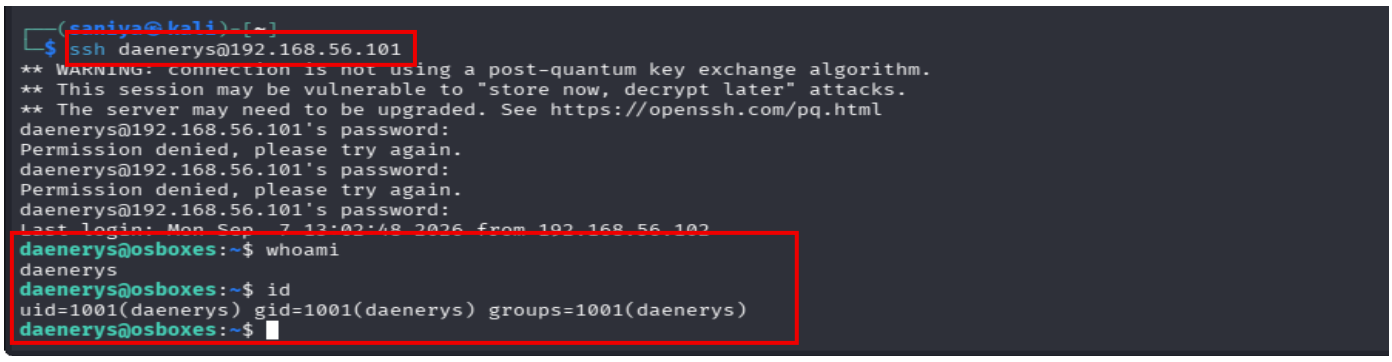


Figure 8: Successful SSH Login as daenerys

Description: With a themed password in hand, SSH was attempted against the target using the obvious matching username, daenerys, and the decoded password, dracarys.

Result: The first two login attempts were rejected with "Permission denied, please try again," but the login succeeded on the third attempt. **whoami** confirmed the session as

daenerys, and id confirmed **uid=1001(daenerys) gid=1001(daenerys) groups=1001(daenerys)** — a normal, low-privileged account.

Interpretation: The username/password pair daenerys / dracarys is valid on this system, giving a genuine low-privilege foothold on the target machine.

Why this matters: This was the pivotal moment of the assessment: access moved from "outside observer" to "authenticated user with a real shell on the box."

Next step: With a shell as daenerys established, the home directory and filesystem were enumerated for further clues toward privilege escalation.

8. Local Enumeration as daenerys

Step 9 — Listing the Home Directory

Command / Action:

```
ls -la
```

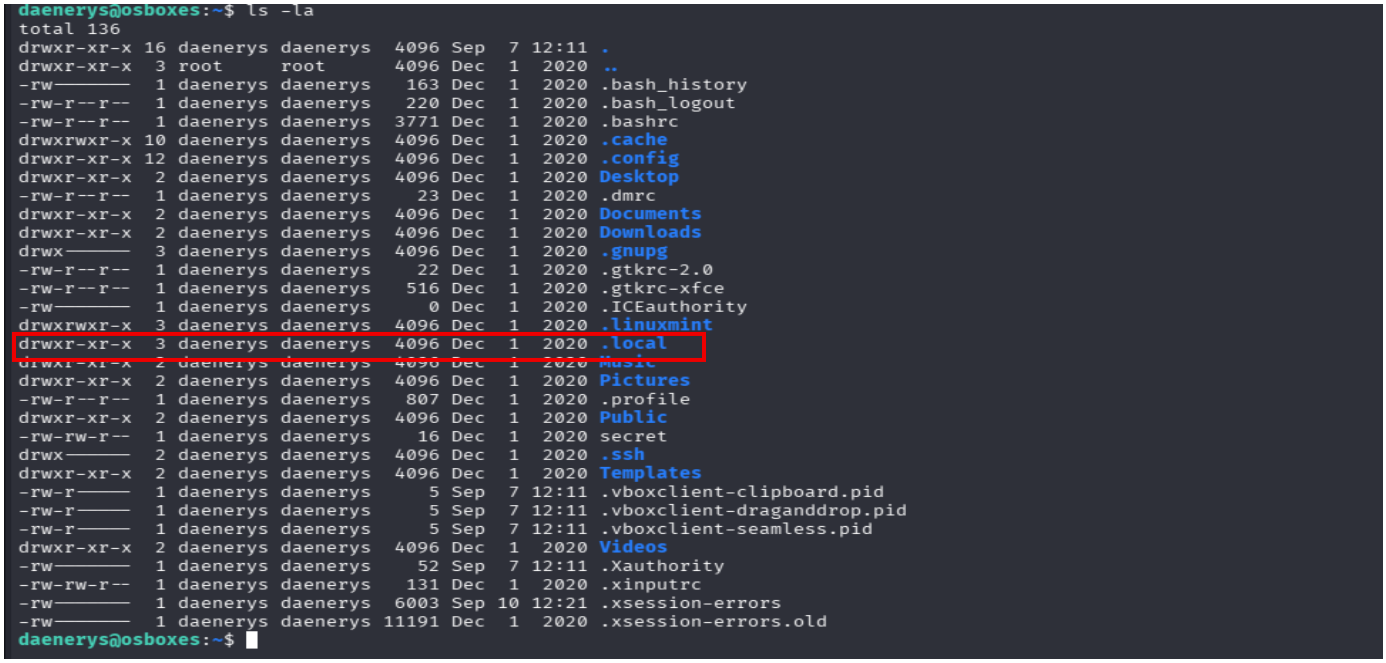


Figure 9: Home Directory Listing for daenerys

Description: A full listing of daenerys's home directory was taken to see what files and folders existed and which might be worth investigating.

Result: Alongside the usual desktop-environment clutter (Desktop, Documents, Downloads, .config, .cache, and **.local** and so on.), two entries stood out: a file named secret (16 bytes, world-readable) and a locked-down .ssh directory (drwx------).

Interpretation: The name secret is an obvious invitation to investigate, and the restrictive .ssh directory hints that key-based material may exist for this or another

account. Nothing here was immediately conclusive, so deeper directories needed to be checked as well.

Why this matters: This gave two more leads to keep in mind while continuing enumeration — a small marked file and a private SSH folder — without yet needing to stop and dead-end on either one.

Next step: Enumeration continued into the hidden application-data folders under the home directory, starting with. .local/

Step 10 — Investigating ~/.local/share

Command :

```
cd .local/
ls
cd share/
ls
ls -la
```

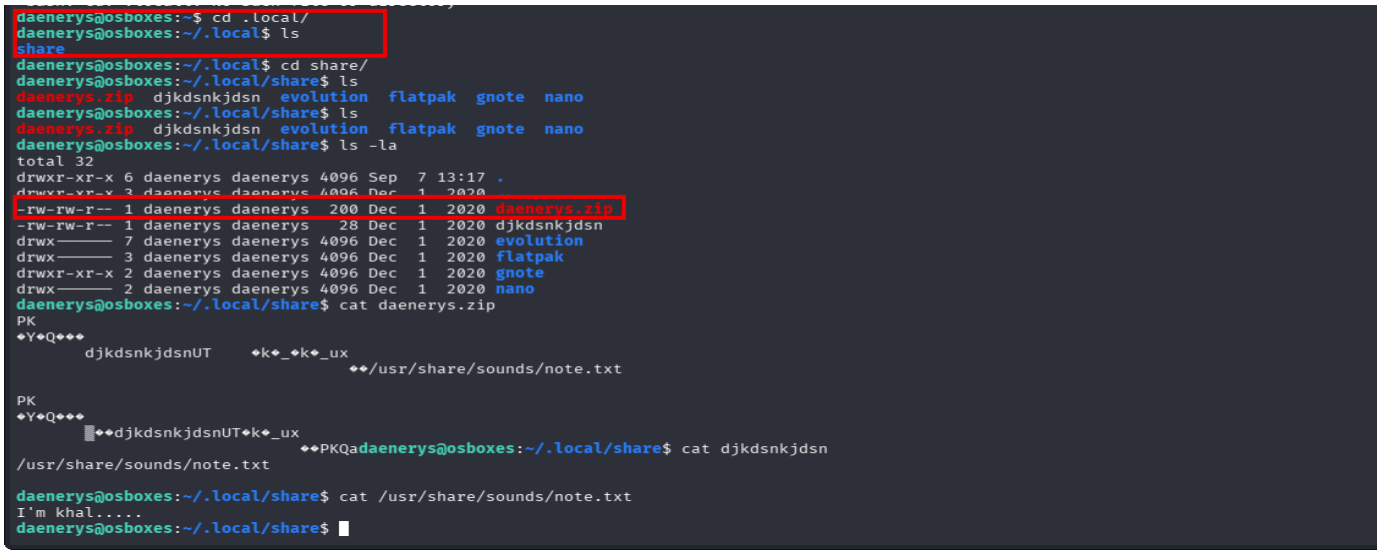


Figure 10: Contents of ~/.local/share, Including daenerys.zip

Description: The ~/.local/share directory (normally used by desktop applications for their own data) was browsed to see if anything had been planted there outside the obvious application folders.

Result: Alongside expected application directories (evolution, flatpak, gnote, nano) were two unusual files: daenerys.zip (200 bytes) and a file with an odd, random-looking name, djkdsnkjdsn (28 bytes).

Interpretation: Two small, out-of-place files sitting next to normal application data folders are very unlikely to be accidental — both were clearly placed here deliberately as part of the challenge's trail of clues.

Why this matters: This gave two very specific artifacts to open next, continuing the same pattern of following breadcrumbs that started with the skeylogger binary.

Next step: Both files were read directly with cat to see what they contained.

Step 11 — Examining daenerys.zip and the djkdsnkjdsn Pointer File

Command / Action:

```
cat daenerys.zip
cat djkdsnkjdsn
```

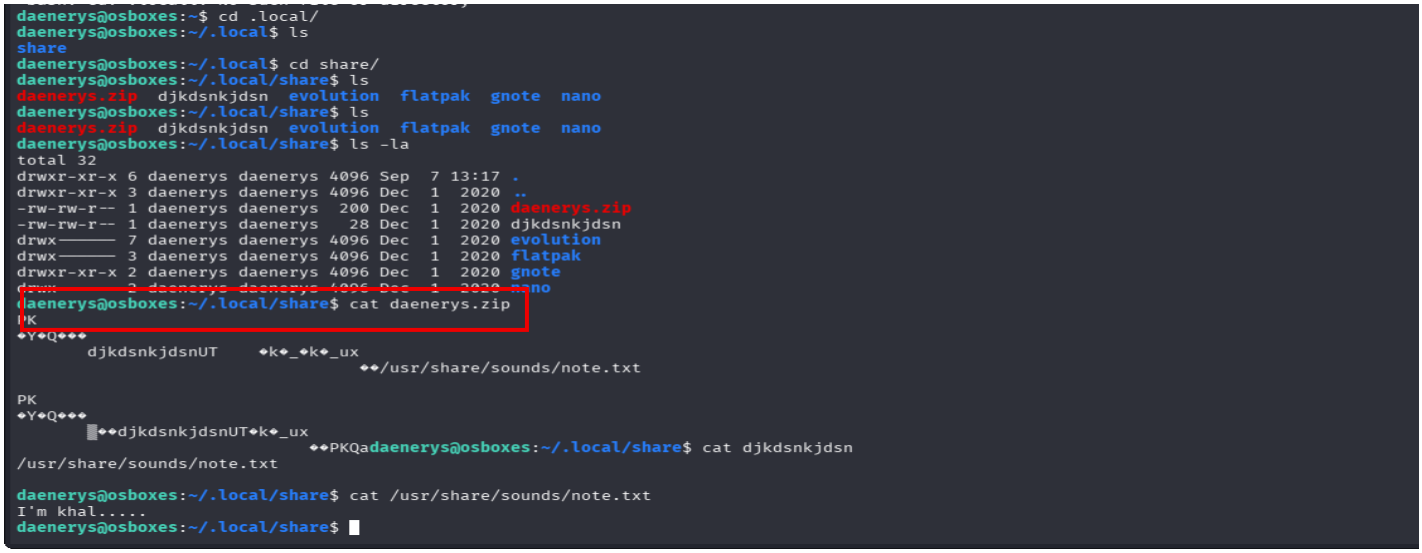


Figure 11: Raw Contents of daenerys.zip and djkdsnkjdsn (same terminal session as Figure 10)

Description: cat daenerys.zip printed the raw bytes of the archive to the terminal, which is not a meaningful way to open a ZIP file but was enough to confirm its file signature. cat djkdsnkjdsn was then used on the oddly named companion file to see if it was plain text.

Result: daenerys.zip began with the standard PK\x03\x04 signature bytes, confirming it genuinely is a ZIP archive (its contents were not further extracted at this stage). djkdsnkjdsn, however, was plain text and contained a single filesystem path: /usr/share/sounds/note.txt.

Interpretation: djkdsnkjdsn is not itself the clue — it is a pointer telling the player exactly which file to read next, tucked away in a system directory that would not normally be checked.

Why this matters: Without this pointer file, the next clue location (/usr/share/sounds/note.txt) would have been very difficult to guess, since it sits well outside the user's home directory.

Next step: The referenced file, /usr/share/sounds/note.txt, was read directly.

Step 12 — Reading the Referenced Clue File

Command / Action:

```
cat /usr/share/sounds/note.txt
```

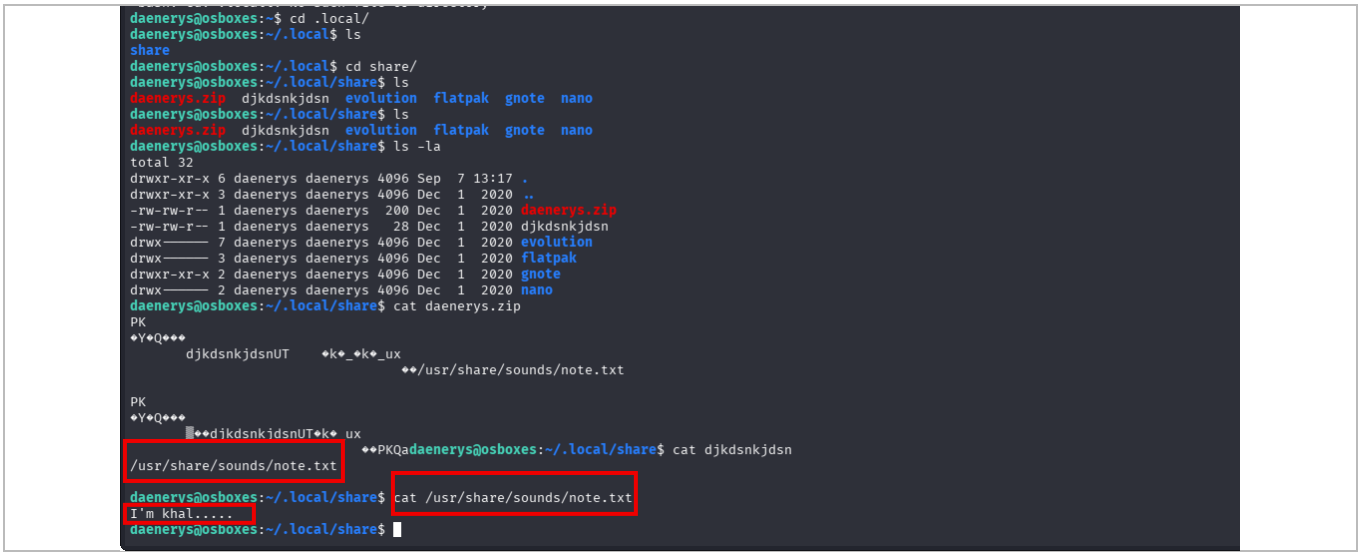


Figure 12: Contents of /usr/share/sounds/note.txt — "I'm khal...." (same terminal session as Figures 10–11)

Description: The path revealed by the pointer file was opened directly.

I'm khal...."

Result: The file contained a short, in-character message: "

Interpretation: This is a narrative clue rather than a technical one. "Khal" is a Game of Thrones/Dothraki title, and the trailing ellipsis strongly implies the reader is meant to work out which specific khal is being referred to.

Why this matters: This shifted the challenge from pure command-line enumeration to open-source research: the next step had to identify exactly who "the khal" was before it could be turned into a usable credential.

Next step: The term "khal" was researched to understand the title, then narrowed down to a specific character.

9. OSINT-Based Clue Research

Step 13 — Researching the Term "Khal"

Command / Action:

```
Google search: who is khal
```

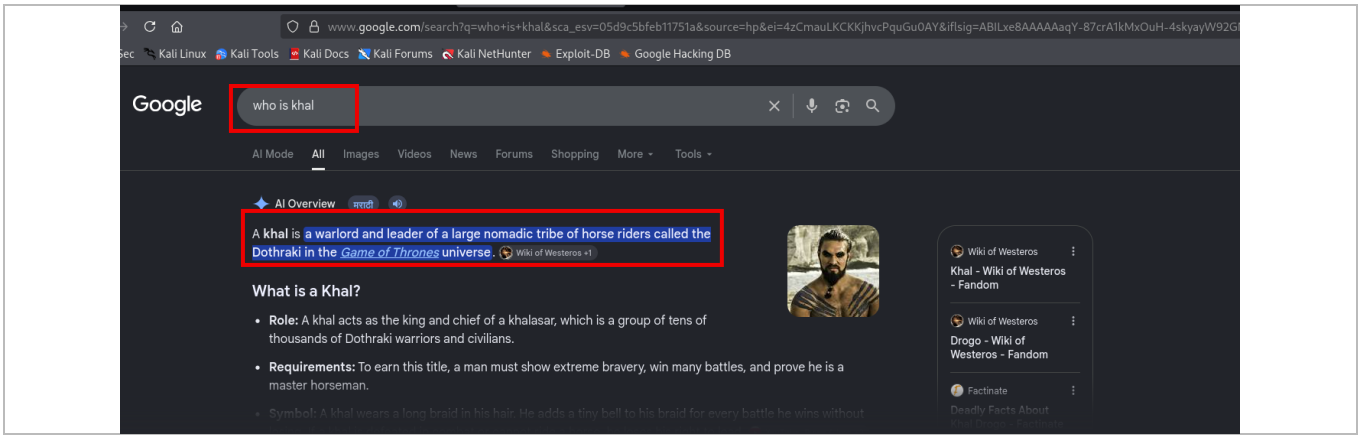


Figure 13: Search Result Explaining the "Khal" Title

Description: A general web search was used to understand what a "khal" actually is within the Game of Thrones/Dothraki universe, since the clue file only used the generic title, not a name.

Result: The search confirmed that a khal is a warlord and leader of a large nomadic tribe of Dothraki horse riders — effectively the king and chief of a khalasar, a group of tens of thousands of Dothraki warriors and civilians.

Interpretation: "Khal" is a title, not a specific person's name, and it is closely tied to Daenerys Targaryen's storyline — reinforcing that the account already compromised (daenerys) and this new clue belong to the same narrative thread.

Why this matters: Confirming khal is a title meant the real answer still had to be a specific, named khal — the search needed to be narrowed further.

Next step: Because Daenerys Targaryen's most famous khal is Khal Drogo, that specific name was searched next.

Step 14 — Identifying the Specific Khal: Khal Drogo

Command / Action:

Google search: who is khaldrogo

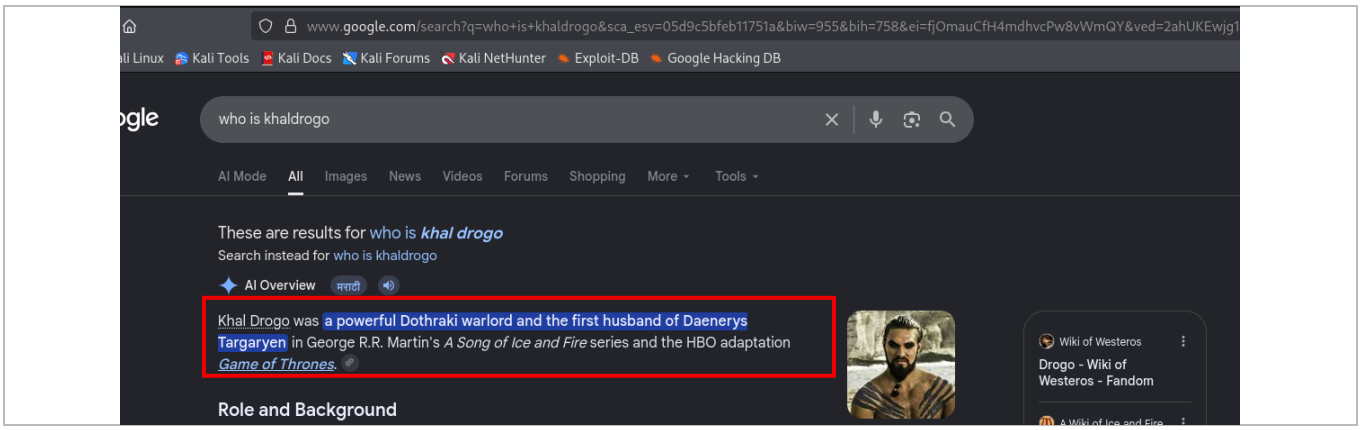


Figure 14: Search Result Identifying Khal Drogo

Description: A follow-up search was run for "khaldrogo," the most iconic khal associated with Daenerys, to confirm the connection before using it as a credential guess.

Result: The search confirmed Khal Drogo was a powerful Dothraki warlord and the first husband of Daenerys Targaryen in George R.R. Martin's A Song of Ice and Fire and its HBO adaptation, Game of Thrones.

Interpretation: This directly answers the "I'm khal...." clue: the note was speaking as Khal Drogo, Daenerys's husband. In a machine already themed around Daenerys and dragons (dracarys), "khaldrogo" is the natural next credential to try — most plausibly as the password for the root account, since root is the system's ultimate "king."

Why this matters: This turned an ambiguous roleplay clue into a specific, testable password candidate for privilege escalation.

Next step: The candidate password khaldrogo was tried against the root account with su.

10. Privilege Escalation to Root and Root Access and Final Flag

Step 15 — Escalating to Root with su

Command / Action:

```
su root
Password: khaldrogo

cd /root

ls

nice.txt
```

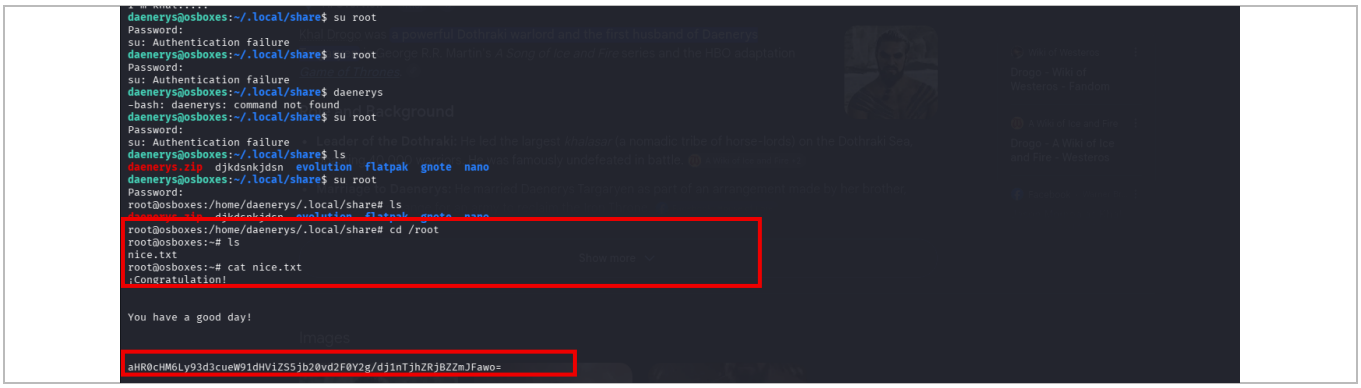


Figure 15: Root Access Obtained After Several su Attempts

Description: su root was run to switch to the root account directly from the daenerys shell. A couple of earlier guesses (visible as "su: Authentication failure" in the terminal

history) were tried first while narrowing down the correct password, along with a mistaken attempt to run daenerys as a command. With an effective root shell, the /root directory — normally accessible only to the root user — was listed and its single file, nice.txt, was read.

Result: The first attempts failed with "su: Authentication failure," but the attempt using khaldrogo succeeded, dropping into a root@osboxes prompt inside /home/daenerys/.local/share.

Interpretation: The research-derived password khaldrogo is the correct root credential for this machine, confirming the intended solution path: skeylogger → dracarys → daenerys (SSH) → hidden clue trail → khaldrogo → root.

Why this matters: This is full privilege escalation — the highest level of access on the system has now been obtained.

12. Credentials and Artifacts Summary

Item	Value	How It Was Obtained
Hidden clue #1	dracarys	Base64 string extracted from strings output of the skeylogger binary
Initial SSH account	daenerys / dracarys	Username inferred from the Game of Thrones theme; password decoded from skeylogger
Pointer file	/usr/share/sounds/note.txt	Referenced inside ~/.local/share/djkdsnkjdsn
In-character clue	"I'm khal...."	Contents of /usr/share/sounds/note.txt
Research finding	Khal Drogo, first husband of Daenerys	OSINT (web search) on "khal" and "khaldrogo"
Root credential	root / khaldrogo	Derived from the OSINT research and confirmed with su
Final flag	¡Congratulation! You have a good day!	/root/nice.txt, read as root

13. Conclusion

The “Who Wants To Be King” machine was fully compromised, from initial network reconnaissance to root access, by following a deliberate trail of clues the machine’s author had planted at each stage. Unlike a purely exploit-driven box, this challenge relied on three main techniques: careful host and service enumeration to find the single exposed file, static analysis of a custom binary to recover a hidden Base64-encoded password, and open-source research to correctly interpret an in-character narrative clue into a second, working credential. Each stage followed the same repeatable pattern — discover a clue, understand what it meant, test a theory based on it, and verify the result before moving on — culminating in a confirmed root shell and the machine’s final congratulatory flag in /root/nice.txt.

Flag: aHR0cHM6Ly93d3cueW91dHViZS5jb20vd2F0Y2g/dj1nTjhZRjZZmJFawo=